

MEDIATEK

MT8676 Android System User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	张超	正式版

目录

版本记录	2
目录	3
图片目录	4
表格目录	4
1 System	5
1.1 Build 系统	5
1.1.1 Build 环境	5
1.1.2 Sync 代码	5
1.1.3 Build 命令	6
1.2 系统概述	6
1.2.1 简单介绍	6
1.2.2 名词解释	7
1.3 架构/流程概述	7
1.3.1 MT8676 Android 软件叠层	7
1.3.2 Bootup	8
1.3.3 Shutdown	14
1.3.4 Android Bootup 日志分析	15
1.4 配置/客制化指南	19
1.4.1 系统相关配置	19
1.5 常见问题/故障排除	22
1.5.1 系统问题需要提供的日志	22
1.5.2 MT8676 平台 DB 分析指引	22
1.5.3 Memory Leak 问题处理方法	32
1.5.4 WDT 简单介绍	33
1.5.5 性能模式	34
1.5.6 CPU 插拔	35
1.5.7 如何抓取 systrace	36
1.5.8 系统常见问题相关参考	36
附件一 附加条款	37

图片目录

图 1-1. MT8676 Android 软件叠层7

图 1-2. Bootup 架构.....8

图 1-3. Boot Rom 主要工作.....9

图 1-4. Bootloader 主要工作10

图 1-5. Kernel C 阶段主要工作12

图 1-6. MT8676 shutdown 流程.....14

图 1-7. DB 的种类.....23

图 1-8. Exception 分类.....24

图 1-9. GAT Tool 主界面25

图 1-10. DBViewer 弹出界面.....25

图 1-11. DB 解析后生成的文件.....25

图 1-12. KE DB 解压后生成的文件26

图 1-13. USBNET 模块导致 KE26

图 1-14. SWT DB 解压后生成文件.....27

图 1-15. system_server 发生异常导致 SWT27

图 1-16. NE DB 解压后生成的文件.....28

图 1-17. systemui 发生异常导致 NE.....29

图 1-18. JE DB 解压后生成的文件.....30

图 1-19. Animation 异常导致 system_server crash 引起 JE.....30

图 1-20. 触发了 MD ASSERT.....31

图 1-21. System UI 发生 ANR31

图 1-22. 如何通过日志快速定位 ANR 问题.....32

图 1-23. cat /proc/meminfo 日志.....33

图 1-24. Kernel memory leak 日志.....33

表格目录

表 1-1. 名词解释.....7

表 1-2. 系统问题需要提供的日志明细.....22

表 1-3. KE 问题需要提供的调试信息.....27

表 1-4. NE 问题需要提供的调试信息29

表 1-5. ANR 问题需要提供的调试信息.....31

1 System

1.1 Build 系统

1.1.1 Build 环境

项目	要求
磁盘空间	不小于 250GB
内存	不小于 24GB
发行版本	Ubuntu 18.04 (推荐)
编译主机需要安装的工具	参考 http://source.android.com/source/initializing.html
Git	1.9.1 或更高版本
Shell	4.4.19 或者更高版本 (Ubuntu18.04 自带)(推荐)
Perl	5.26.1 (Ubuntu18.04 自带)(推荐)
Python	Python 2.7.17 & Python 3.6.9 (Ubuntu18.04 自带)(推荐)
GNU Make	4.1 (Ubuntu18.04 自带)(推荐)

1.1.2 Sync 代码

Step1: 创建一个目录用于存放 code

命令: `mkdir <working folder> && cd <working folder>`

Step2: Codebase 初始化

命令: `repo init -u https://git01.mediatek.com/alps_release/platform/manifest -b <branch> -m <manifest> --no-repo-verify`

<branch>: branch 名称

<manifest>: 指定的 branch 和版本

场景 1: init latest code

```
$repo init -u https://git01.mediatek.com/alps_release/platform/manifest -b $company -m $(release_branch)-default.xml --no-repo-verify
```

场景 2: init specific version

```
$repo init -u https://git01.mediatek.com/alps_release/platform/manifest -b $company -m $(release_tag).xml --no-repo-verify
```

Step3: Code 抓取

场景 1: Sync full codebase

```
$repo sync -c -f -j8 --no-repo-verify --optimized-fetch
```

场景 2: Sync 单一 repo

```
$repo sync <repository name> /*<repository name> can be repository name or repository path */
```

例: \$repo sync device/common

场景 3: Sync 多个 repo

```
$repo sync <repository name1 repository name2> /*repository1 repository2 separated by space */
```

例: \$repo sync platform/art platform/abi/cpp

1.1.3 Build 命令

MT8676 使用 LD2.0 build, 请参考 MOL: <https://online.mediatek.com/apps/quickstart/QS00266>。

Build 命令:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py -run full_${Project}-${mode}
```

例:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py -run full_auto8676p1_64_bsp-userdebug
```

输出: 图片生成目录 out/target/product/\${project}/merged

LD2.0 会产生 out_sys & out_hal & out_krn & out 这些文件夹用来存放每个叠层的图片。

详情请参考: <https://online.mediatek.com/apps/quickstart/QS00178>。

查询 Build 命令:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_${Project}-${mode}
```

1.2 系统概述

1.2.1 简单介绍

本章节主要介绍 MT8676 系统相关内容以及常见系统问题的处理方法。

1.2.2 名词解释

表 1-1. 名词解释

缩略词	全称及释义
AED	Android Exception Daemon 异常守护程序
JE	Java Exception Java 异常
NE	Native Exception Native 异常
KE	Kernel Exception 内核异常
EE	External Exception 外部异常
ANR	Application Not Response 应用程序无响应
SWT	Software Watchdog Timeout 软件看门狗超时

1.3 架构/流程概述

1.3.1 MT8676 Android 软件叠层

MT8676 Android 软件叠层如图 1-1 所示（需要放大才能看清楚）。

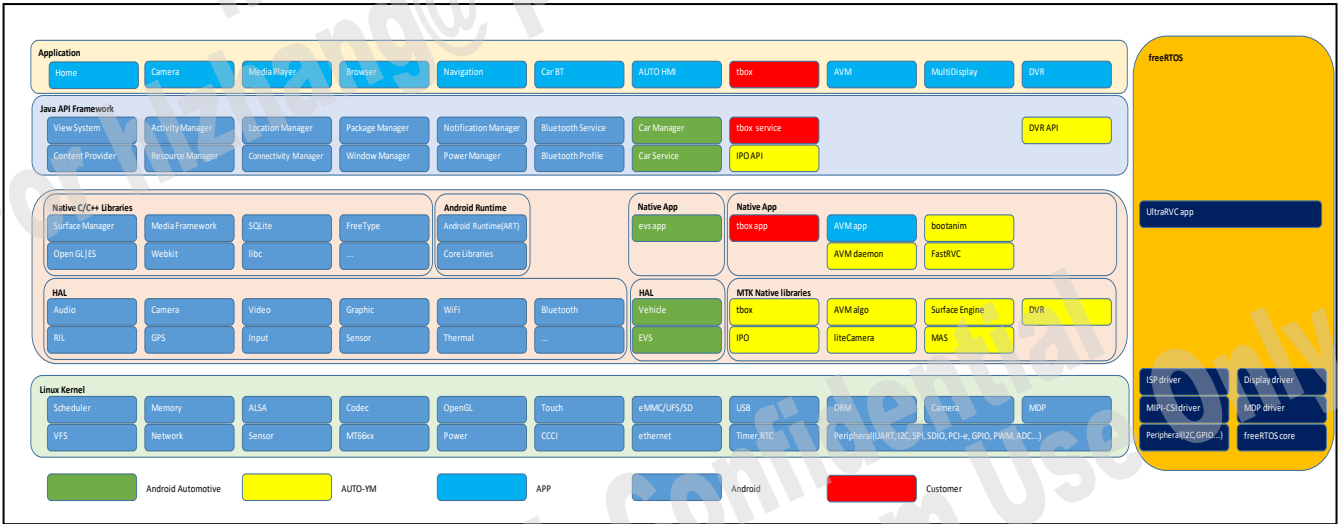


图 1-1. MT8676 Android 软件叠层

1.3.2 Bootup

1.3.2.1 Bootup 架构

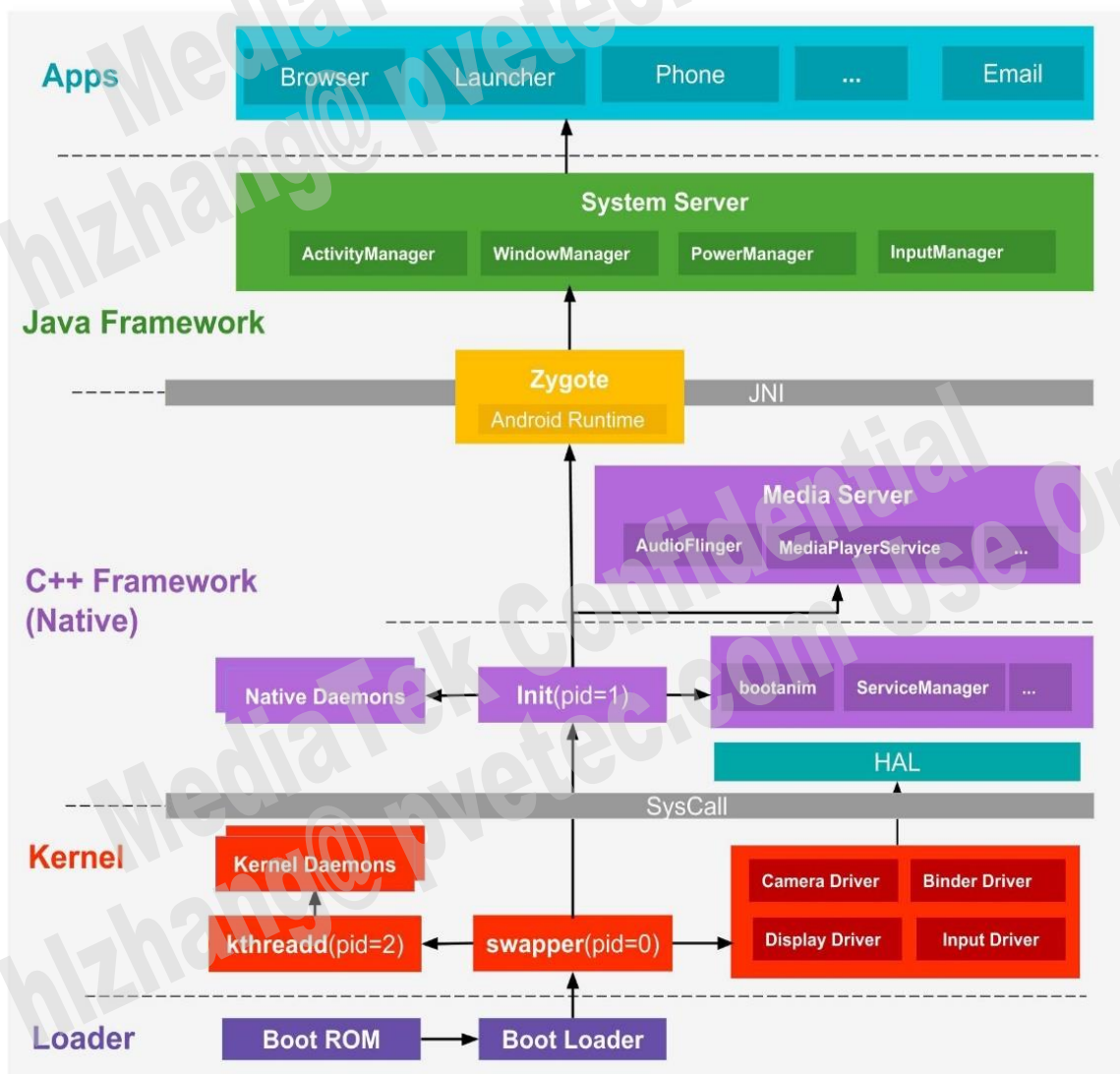


图 1-2. Bootup 架构

- 第一步 loader 层：上电之后，先执行 bootrom 中的 bootcode，将 bootloader 从外部 flash 加载到 SRAM 中，对 DRAM 进行初始化，拉起 Linux kernel。Boot Loader 分为 preloader 和 LK 两大部分，preloader 运行在 SDRAM 中，LK 运行在 DRAM 中，LK 会去加载 kernel 程序。
- 第二步 kernel 层：首先启动 0 号进程，用于初始化进程管理、内存管理、加载驱动程序；再 fork 出 2 号进程，2 号进程是所有 Linux 内核进程的鼻祖。
- 第三步 native 层：kernel 层的驱动程序加载完毕之后，硬件设备驱动与 HAL 层进行交互，启动 1 号进程。

- 第四步 native 层：init 进程（1 号进程）启动之后会启动各种用户守护进程，并且会启动 servicemanager（binder 服务管家）等重要服务，同时解析对应的配置文件 init.rc 并且 fork 出 Zygote 进程。
- 第五步：Zygote 进程是介于 native 层与 Java 层的进程，它会加载虚拟机 JVM，注册 JNI 函数，打通 native 到 Java 层的通道，并且启动 systemserver（负责整个 Java 框架）以及第一个应用程序进程启动器。

1.3.2.2 Boot ROM

1.3.2.2.1 简单介绍

Boot Rom 是一个硬件设备（类似于 SRAM/DRAM），里面存着一段出厂自带的不可修改的已经固化的程序 bootcode。上电之后，Boot Rom 开始执行内部事先设定好的 bootcode，加载引导程序 preloader 到 SRAM 中然后执行。

1.3.2.2.2 主要工作

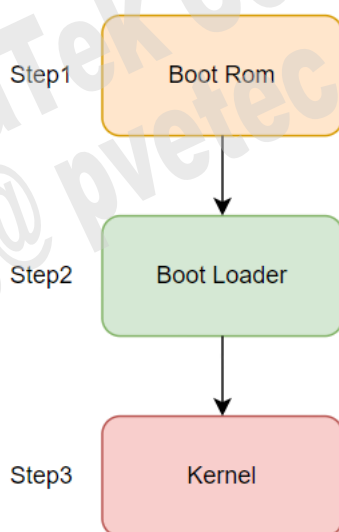


图1-3. Boot Rom 主要工作

1. 加载引导程序 preloader 到 SRAM 中然后执行；
2. 当检测不到有效的镜像或者代码时，Boot Rom 引导进入下载模式；
3. 执行一些校验工作；
4. 硬件初始化，串口，flash 等。

1.3.2.3 Bootloader

1.3.2.3.1 简单介绍

Bootloader 分为 preloader 和 LK (Little Kernel 一个开源的 bootloader 项目, 但是只支持 Arm 和 x86 平台, 其显著的特点为实现了一个简单的线程机制) 两大部分。

Preloader 存储在 Flash 中, 运行在 SRAM (SRAM 不需要初始化) 中, preloader 可以直接被 bootrom 加载到 SRAM 中运行, 主要作用是初始化 DRAM。

LK 运行在 DRAM 中, LK 的主要作用是加载 kernel 到 DRAM 中, LK 同样也储存在 Flash 中。

1.3.2.3.2 主要工作

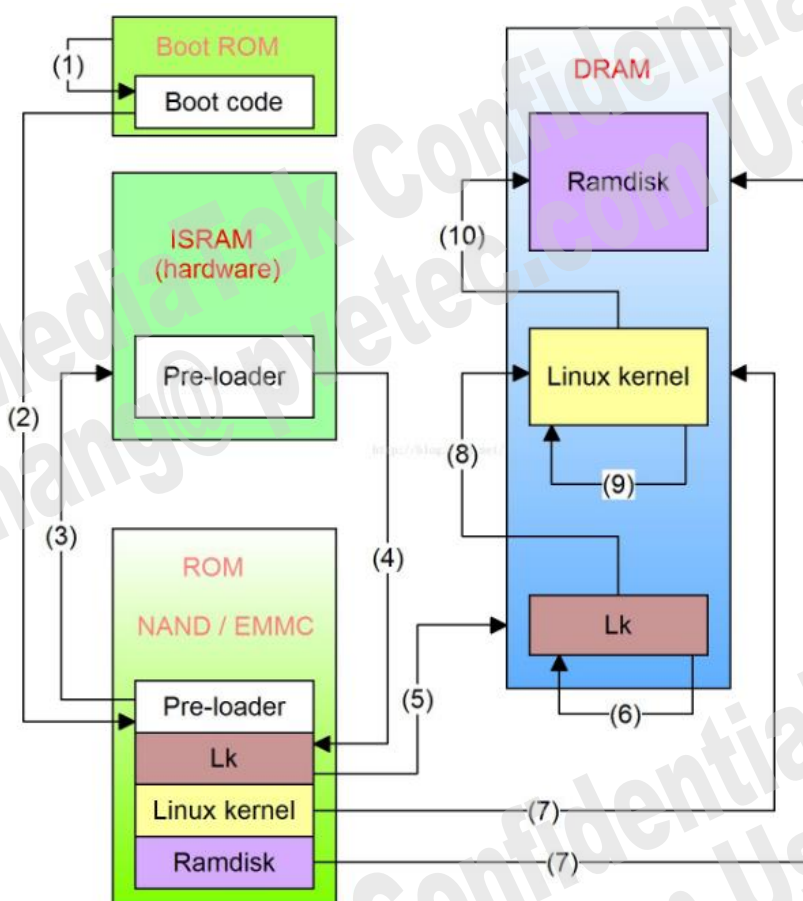


图 1-4. Bootloader 主要工作

1. 上电之后先执行 BootRom 里面的 BootCode。
2. BootCode 在 Flash 中定位到 preloader。

3. 将 preloader 从 Flash 中加载到 SRAM 中（0x00200F10），开始初始化外部 DRAM。
4. Preloader 在 Flash 中定位到 LK。
5. 将 LK 从 Flash 中加载到外部的 DRAM 运行。
6. LK 自身初始化。
7. LK 将 boot.image/vendor_boot.img 解压为 kernel 和 ramdisk，从 Flash 中加载到 DRAM 中。
8. LK 初始化 kernel。
9. 执行 kernel 部分程序（kernel 初始化完成后 fork 出 init 进程）。
10. 拉起 ramdisk 中的 init 程序，进入用户空间初始化，init 进程 fork 出 Zygote 进程，....，直到整个 Android 启动完成。

1.3.2.4 Kernel

1.3.2.4.1 简单介绍

内核（Kernel）是操作系统的核心组件，它负责管理计算机硬件和软件资源，提供操作系统和应用程序的基本服务。内核是操作系统的第一个加载，并作为整个操作系统的核心运行在内存中，控制着所有系统资源，如 CPU、内存、输入/输出、文件系统等。内核提供了底层的服务和接口，供应用程序调用和使用。

1.3.2.4.2 主要工作

Kernel 汇编阶段（由 ENTRY(_stext)开始）

1. 设置为 SVC 模式，关闭所有中断。
2. 获取 CPU ID。
3. 验证 dtb。
4. 创建页表项。
5. 配置 r13 寄存器，也就是设置打开 MMU 之后要跳转到的函数。
6. 使能 MMU。
7. 跳转到 start_kernel，进入 C 阶段。

Kernel C 阶段（由 kernel_init 开始）

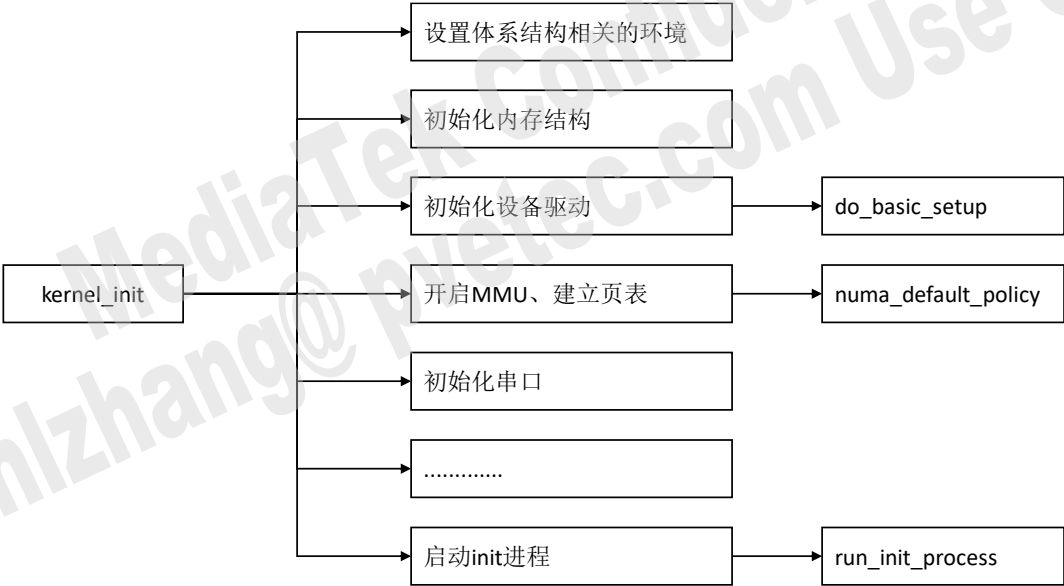


图1-5. Kernel C 阶段主要工作

主要完成硬件平台相关的初始化工作，在相关初始化结束之后执行 init 程序。

1.3.2.5 init 进程

1.3.2.5.1 简单介绍

init 进程是 Linux 系统中用户空间的第一个进程，进程号为 1。

当 bootloader 启动后，启动 kernel，kernel 启动之后，在用户空间启动 init 进程，再通过 init 进程，来读取 init.rc 中的相关配置，从而启动其他相关进程。

1.3.2.5.2 主要工作

- 分析 init.rc 启动脚本文件，根据文件内容执行相应的功能 ss。
- 当一些关键进程死亡时，重启该进程。
- 提供 Android 系统的属性服务。

1.3.2.5.3 代码介绍

Init 的入口函数位于 /system/core/init/init.c。

1.3.2.6 Zygote

1.3.2.6.1 简单介绍

当 Android 设备启动时，系统首先启动 Zygote 进程。Zygote 进程会预加载和初始化一些常用的系统库和资源文件等，这样可以加快其他应用程序的启动速度，因为这些资源已经被加载到内存中。

1.3.2.6.2 主要工作

ZygoteInit 类的 main 主要完成以下五个工作。

1. registerZygoteSocket(): Zygote 进程注册监听 socket。
2. preload(): 加载常用的 Java 类和系统资源。
3. startSystemServer(): 启动 SystemServer 进程。
4. runSelectLoopMode(): 进入循环监听模式。
5. closeServerSocket(): 进程退出时，关闭 socket 监听。

1.3.2.6.3 代码介绍

Zygote 入口函数位于 kernel/sched/fork.c。

1.3.2.7 SystemServer

1.3.2.7.1 简单介绍

SystemServer 是由 ZygoteInit 的 main 创建的，调用 caller.run 之后会执行 SystemServer 的 main 函数。

1.3.2.7.2 主要工作

- 管理系统级别服务：SystemServer 管理许多 Android 系统的核心服务和功能，如 Activity Manager Service、Content Provider Service、Package Manager Service、Notification Manager Service 等等。它们提供了必要的基础设施，供应用程序运行和交互。

- 管理系统资源：SystemService 通过资源管理器（Resource Manager）来管理系统资源，包括内存、进程、线程、文件系统、网络等等。资源管理器可以分配和回收系统资源，以确保系统的稳定性和安全性。
- 提供系统级别设置和配置：SystemService 提供了系统级别的设置和配置，如网络设置、音量设置、时间设置、语言设置等等。这些设置是系统级别的，会影响整个系统的表现和操作方式。

1.3.2.7.3 代码介绍

SystemService 入口函数位于 `frameworks/base/services/java/com/android/server/SystemService.java`。

1.3.3 Shutdown

1.3.3.1 Shutdown 流程图



图1-6. MT8676 shutdown 流程

- 调用 `ShutdownThread.shutdown()`。
 1. 获取用户关机行为。
 2. 注册关机广播。
 3. 创建关机对话。
 4. 执行一系列关机流程。

- 出现关机对话时，若用户确认关机，则会执行 `beginShutdownSequence` 开始关机流程。
 - 显示关机进度条对话框。
 - 启动 `ShutdownThread` 线程，执行 `run` 方法。

1.3.3.2 ShutdownThread 主要作用

- 发送有序的关机广播。
- 测量关闭某些项目的时间。
- 关闭 `ActivityManager`。
- 关闭 `PackageManager`。
- 关闭 `Radios`。
- 完成 `SystemService` 的关闭。
- 初始化并判定执行关机还是重启。

如果有开 IPO 功能，则进 IPO，然后休眠。

1.3.4 Android Bootup 日志分析

```
[14:40:10:408] Pll init start... //进入程序的第一句 log, 标志着程序启动, 进入 perloader
...
[14:40:10:444] Pll init Done!!
[14:40:10:444] #T#PLL=24
[14:40:10:445] #T#GPIO=0
[14:40:10:445] [RGU] rst from: kernel
[14:40:10:445] drm_latch_en: MTK_DRM_LATCH_EN(95027EF0)
[14:40:10:445] [RGU] STA from reg: 0x40000000 //RGU 打印, 可以关注系统是否异常重启
[14:40:10:446] [RGU] MODE: 0x25
[14:40:10:446] [RGU] STA: 0x40000000
[14:40:10:446] [RGU] LENGTH: 0x1FFFE0
[14:40:10:446] [RGU] INTERVAL: 0xFFFF
[14:40:10:447] [RGU] SWSYSRST: 0x0
[14:40:10:447] [RGU] LATCH_CTL: 0xF21E79
[14:40:10:447] [RGU] LATCH_CTL2: 0x0
[14:40:10:447] [RGU] NONRST_REG: 0x0
[14:40:10:448] [RGU] NONRST_REG2: 0x2C002000
[14:40:10:448] [RGU] REQ_MODE: 0xFD00E3
[14:40:10:448] [RGU] REQ_IRQ_EN: 0x7D0065
[14:40:10:448] [RGU] DEBUG_CTL: 0x0
[14:40:10:448] [RGU] parse g_rgu_status: 2 (0x2)
```

```

[14:40:10:449] [RGU] Set NONRST_REG to 0x40000000
[14:40:10:449] [RGU] mtk_wdt_mode_config mode value=34, tmp:22000034
...
[14:40:10:541] [PMIC] PONSTS[0xC]=0x0 //PMIC register 打印, 可以关注 PMIC 行为是否有异常
[14:40:10:541] [PMIC] POFFSTS[0xD]=0x400
[14:40:10:541] [PMIC] POFFSTS2[0xF]=0x0
[14:40:10:542] [PMIC] POFFSTS4_5[0x11]=0xFFFF
[14:40:10:542] [PMIC] POFFSTS6[0x13]=0xF
[14:40:10:542] [PMIC] VR_SDN_MODE0_1[0x214]=0x0
[14:40:10:542] [PMIC] VR_SDN_MODE2_3[0x216]=0x0
[14:40:10:543] [PMIC] VR_SDN_MODE4_5[0x218]=0x0
[14:40:10:543] [PMIC] VR_SDN_MODE6_7[0x21A]=0x0
[14:40:10:547] [PMIC] STS_THR_LOC[0x10]=0x0
[14:40:10:547] [PMIC] STRUP_CON4[0xA1A]=0x0
[14:40:10:548] [PMIC] STRUP_CON12[0xA0F]=0x1
[14:40:10:548] [PMIC] WDTRSTB[0x139]=0x4
[14:40:10:548] [pmic_check_rst] AP Watchdog
...
[14:40:12:051] [Calibration Summary] Frequency 3750 //DRAM 各频点 calibration
[14:40:12:051] CH 0, Rank 0
[14:40:12:051] SW Impedance : PASS
[14:40:12:051] 8 Phase : FAST K
[14:40:12:051] DUTY Scan : FAST K
[14:40:12:051] ZQ Calibration : PASS
[14:40:12:052] Jitter Meter : NO K
[14:40:12:052] CBT Training : NO K
[14:40:12:052] Write leveling(PI) : FAST K
[14:40:12:052] Write leveling(DLY) : FAST K
[14:40:12:052] DUTY CYCLE MONITOR : FAST K
[14:40:12:052] RX DQS gating : FAST K
[14:40:12:052] RX OFFC : NO K
[14:40:12:053] RX DQ/DQS(RDDQC) : FAST K
[14:40:12:053] RX DQ/DQS(RDDQC_DQM_ONLY) : FAST K
[14:40:12:053] TX DQ/DQS : PASS
[14:40:12:053] RX DATLAT : FAST K
[14:40:12:053] RX DQ/DQS(Engine) : FAST K
[14:40:12:053] RX RDQS DCA : FAST K
[14:40:12:054] RX RDQS DCC : FAST K
[14:40:12:054] TX OE : NO K
[14:40:12:054] All Pass.
...
[14:40:12:892] [BLDR] Starting tool handshake. //USB 和 flashtool USB 握手, 如果成功则加
载 DA 进入 Download Mode
[14:40:15:971] #T#UART handshake init=1
...
[14:40:16:098] [PART] load "lk_a" from 0x000000003B8DB9C0 (dev) to 0x78000000 (mem)
[SUCCESS] //Preloader 加载相关 image 到 DRAM
[14:40:16:098] [PART] load speed: 155765KB/s, 638016 bytes, 4ms
[14:40:16:098] [PART] img vfy...[SEC] img auth ok
...
[14:40:16:418] welcome to lk //进入 LK BL2_EXT

```

```

[14:40:16:418]
[14:40:16:418] lk variant: BL2_EXT
...
[14:40:19:081] Say Hello from EL2!
[14:40:19:082]
[14:40:19:082] MIDR: 411fd461, MPIDR: 81000000
[14:40:19:082] CPU Core: 0!
[14:40:19:082] GZ Platform: MTK_ARMV8
[14:40:19:082] Memory base: 0xbe5400000, size: 0x2a00000
[14:40:19:083] lk_init 0x38082728-0x38082ba8=1152
[14:40:19:083] 48600000 48600000 8000 11001000 in lk_main
[14:40:19:083] ==> GZ (0) [ 0.002364] Version of GZ-CORE: GZ_CORE_hypervisor:
3.2.0.039.U0MP1, Built: 11:53:14 Sep 6 2023
[14:40:19:083] ==> GZ (0) [ 0.003564] platform_init_mmu_mappings membase: 0xbe5400000
[14:40:19:084] ==> GZ (0) [ 0.004372] platform_init_mmu_mappings memsize: 0x2a00000
[14:40:19:084] ==> GZ (0) [ 0.005158] platform_init_mmu_mappings kernel offset:
0x38000000
[14:40:19:084] ==> GZ (0) [ 0.006019] platform_init_mmu_mappings ram phys: 0xcld400000
[14:40:19:085] ==> GZ (0) [ 0.006838] platform_init_mmu_mappings ram size: 2a00000
[14:40:19:085] ==> GZ (0) [ 0.007740] INIT: cpu 0, calling hook 0x3800f0f0
(libvmm_early) at level 0x10000, flags 0x1
[14:40:19:085]
[14:40:19:085] welcome to lk/MP... //进入 GZ
...
[14:40:20:982] welcome to lk //进入 LK
[14:40:20:995]
[14:40:20:995] lk variant: BL33
[14:40:20:995] boot args 0x48600000 0x0 0x0 0x0
[14:40:20:995] version:
[14:40:20:995] arch: arm64
[14:40:20:995] platform: mediatek
[14:40:20:995] target: auto8676p1_64_bsp
[14:40:20:995] project: auto8676p1_64_bsp
[14:40:20:997] buildid: d130cca93e_202407032103
...
[14:40:25:919] [PROFILE] :: hyp_unmap2() takes 1376 ms
[14:40:25:925] [PROFILE] :: name: pl_t, time: 6106 ms
[14:40:25:925] [PROFILE] :: name: logo_t, time: 1533 ms
[14:40:25:925] [PROFILE] :: name: lk_t, time: 5048 ms
[14:40:25:929] [PROFILE] :: name: bl2_ext_t, time: 1842 ms
[14:40:25:929] [PROFILE] :: name: tfa_t, time: 644 ms
[14:40:25:929] [PROFILE] :: name: sec_os_t, time: 0 ms
[14:40:25:929] [PROFILE] :: name: gz_t, time: 1860 ms
[14:40:25:934] boot_linux_fdt:543: booting linux @ 0x40000000, ramdisk @ 0x66f00000
(31301139) dtb @ 0x47c80000 (487064)
[14:40:25:934] boot_linux_fdt:544: lk boot mode = 0
[14:40:25:934] boot_linux_fdt:546: lk finished --> jump to linux kernel 64Bit //LK
jumps to kernel
...
[14:40:26:122] [ 0.000000] [ T0] swapper: [name:setup&] Booting Linux on physical CPU
0x0000000000 [0x411fd461] //kernel 启动第一行打印

```

```

[14:40:26:122] [ 0.000000][ T0] swapper: [name:main&]Linux version 6.1.78-android14-
11-maybe-dirty (build-user@build-host) (Android (10087095, +pgo, +bolt, +lto, -mlgo, based
on r487747c) clang version 17.0.2 (https://android.googlesource.com/toolchain/llvm-project
d9f89f4d16663d5012e5c09495f3b30ece3d2362), LLD 17.0.2) #1 SMP PREEMPT Thu Jan 1 00:00:00
UTC 1970
[14:40:26:124] [ 0.000000][ T0] swapper: [name:setup&]Machine model: MT8676
[14:40:26:124] [ 0.000000][ T0] swapper: [name:stackdepot&]Stack Depot is disabled
[14:40:26:124] [ 0.000000][ T0] swapper: [name:kvm&]KVM is not available. Ignoring
kvm-arm.mode
[14:40:26:124] [ 0.000000][ T0] swapper: earlycon: [name:earlycon&]uart8250 at
MMIO32 0x0000000011001000 (options '')
...
[14:40:30:574] [ 3.387865][T700001] init: mtk-smi-larb 1a02c000.smi-larb25:
[name:mtk_smi&]Succeed to get smi-comm dev for mmqos //init 加载 ramdisk ko
[14:40:30:574] [ 3.389412][T700075] kworker/7:1: [name:mtk_vmm_spm&][ISPDVFS]
regulator_event_notify(): VMM regulator before disable, ver = 3
[14:40:30:575] [ 3.389412][T700075]
[14:40:30:575] [ 3.391773][T700001] init: [name:mtk_vmm_spm&][ISPDVFS]
regulator_event_notify(): VMM regulator enable done, ver = 3
[14:40:30:576] [ 3.391773][T700001]
[14:40:30:576] [ 3.393960][T700001] init: mtk-smi-larb 1a02d000.smi-larb26:
[name:mtk_smi&]Succeed to get smi-comm dev for mmqos
[14:40:30:576] [ 3.395507][T700075] kworker/7:1: [name:mtk_vmm_spm&][ISPDVFS]
regulator_event_notify(): VMM regulator before disable,
...
[14:40:36:929] [ 8.972200][T1400001] init: init 23: Parsing file
/system/etc/init/hw/init.rc... //init 解析 rc 文件
[14:40:36:937] [ 8.974057][T1600001] init: init 23: Added '/init.environ.rc' to import
list
[14:40:36:939] [ 8.974965][T1600001] init: init 23: Added
'/system/etc/init/hw/init.usb.rc' to import list
[14:40:36:945] [ 8.976021][T1600001] init: init 23: Added '/init.mt8676.rc' to import
list
[14:40:36:965] [ 8.976902][T1600001] init: init 23: Added
'/vendor/etc/init/hw/init.mt8676.rc' to import list
[14:40:36:965] [ 8.978016][T1600001] init: init 23: Added
'/system/etc/init/hw/init.usb.configfs.rc' to import list
[14:40:36:965] [ 8.979172][T1600001] init: init 23: Added
'/system/etc/init/hw/init.zygote64.rc' to import list
[14:40:36:975] [ 8.980525][T1600001] init: init 23: Added
'/system/etc/init/hw/init.boringssl.zygote64.rc' to import list
[14:40:36:976] [ 8.982166][T1600001] init: init 23: Parsing file /init.environ.rc...
[14:40:36:977] [ 8.983054][T1600001] init: init 5: Parsing file
/system/etc/init/hw/init.usb.rc...
[14:40:36:978] [ 8.984895][T1600001] init: init 5: Parsing file /init.mt8676.rc...
[14:40:36:978] [ 8.985735][T1600001] init: init 5: Unable to read config file
'/init.mt8676.rc': open() failed: No such file or directory
...
[14:40:42:042] [ 14.402919][T1400415] modprobe: mt6338-efuse mt6338-efuse:
[name:nvmem_mt6338_efuse&]EFUSE[58]=0x83 //加载
vendor ko

```

```
[14:40:42:042] [ 14.404272][T1400415] modprobe: mt6338-sound mt6338-sound:
[name:snd_soc_mt6338&]mt6338_codec_init_reg()
[14:40:42:042] [ 14.417048][T1400415] modprobe: snd-scp-ultra snd-scp-ultra:
[name:snd_soc_mtk_scp_ultra&]mtk_scp_ultra_pcm_new()
[14:40:42:043] [ 14.418416][T1400415] modprobe: snd-scp-ultra snd-scp-ultra:
[name:snd_soc_mtk_scp_ultra&]mtk_scp_ultra_reserved_dram_init(), sce reserve mem
pa=0x8f835700, va=0xffffffffc018835700, size=0x1a000
[14:40:42:043] [ 14.420562][T1400415] modprobe: snd-scp-ultra snd-scp-ultra:
[name:snd_soc_mtk_scp_ultra&]mtk_scp_ultra_reserved_dram_init(), dump pa=0x8f835700,
va=0xffffffffc018835700, size=0x1a000
...
[14:40:47:477] [ 20.251333][T1100322] init: init 29: [20203][1012]Wait for property
'vendor.all.modules.ready=1' took 6885ms
[14:40:47:477] [ 20.252841][T1700324] init: [name:bootprof&]BOOTPROF:
20252.839124:modprobe: Load_Module_DONE //ko 加载完成
[14:40:47:477] [ 20.254094][T1700001] init: init 25: [20211][5676]Lastest epoll wait
took 5676ms
[14:40:47:485] [ 20.255294][T1700001] init: init 25: [20213][0]Command 'write
/proc/sys/vm/watermark_boost_factor 0' action=post-fs-data
(/vendor/etc/init/hw/init.mt8676.rc:178) took 0ms and failed: Unable to write to file
'/proc/sys/vm/watermark_boost_factor': open() failed: Permission denied
[14:40:47:488] [ 20.258432][T1700001] init: init 25: [20213][0]Service 'insmod_sh' (pid
387) exited with status 0 oneshot service took 10.190000 seconds in background
[14:40:47:488] [ 20.260124][T1700001] init: init 25: [20213][0]Sending signal 9 to
service 'insmod_sh' (pid 387) process group...
...
[14:40:48:412] [ 21.214884][T1500324] init: [name:bootprof&]BOOTPROF:
21214.882203:INIT:zygote-start //启动 zygote 进程
[14:40:48:413] [ 21.216010][T1500001] init: init 25: [21174][0]processing action
(firmware_mounts_complete) from (/system/etc/init/hw/init.rc:521)
[14:40:48:413] [ 21.217542][T1500001] init: init 25: [21175][0]processing action (early-
boot) from (/vendor/etc/init/hw/init.mt8676.rc:751)
[14:40:48:413] [ 21.219106][T1500324] init: [name:bootprof&]BOOTPROF:
21219.105049:INIT:early-boot //进入 early boot 阶段, 负责一些基础服务和参数的设置
...
[14:40:55:318] [ 28.131865][T1700324] init: [name:bootprof&]BOOTPROF: 28131.864681:
OFF (KO:356) //开机完成
```

1.4 配置/客制化指南

1.4.1 系统相关配置

1.4.1.1 快速开机配置

Preloader:

```
vendor/mediatek/proprietary/bootable/bootloader/preloader/custom/auto8676p1_64_bsp/auto8676p1_64_bsp.mk
```

```
+ MTK_SEC_USBDL=ATTR_SUSBDL_DISABLE
+ MTK_SEC_BOOT=ATTR_SBOOT_DISABLE
---
- MTK_SEC_USBDL=ATTR_SUSBDL_ONLY_ENABLE_ON_SCHIP
- MTK_SEC_BOOT=ATTR_SBOOT_ENABLE
15c15
+ MTK_SECURITY_SW_SUPPORT=no
---
- MTK_SECURITY_SW_SUPPORT=yes
58,71c58
+ MTK_CPU_BOOST = yes
+ MTK_REDUCE_DRAM_FREQ = yes
+ ifeq ("$(TARGET_BUILD_VARIANT)", "user")
+     CFG_USB_TOOL_HANDSHAKE := 0
+     CFG_LOG_LEVEL := 0
+     CFG_LOG_STORE_SUPPORT := 0
+     CFG_ATF_LOG_SUPPORT := 0
+     CFG_DOE_CONFIG_ENV_SUPPORT := 0
+ endif
+ ifeq ("$(TARGET_BUILD_VARIANT)", "userdebug")
+     CFG_USB_TOOL_HANDSHAKE := 1
+     CFG_LOG_LEVEL := 2
+ endif
```

LK2:

```
vendor/mediatek/proprietary/bootable/bootloader/lk2/project/auto8676p1_64_bsp.mk
```

```
27c27
+ MTK_LCM_LIST_SUPPORT="jd9365da_wxga_dsi_vdo"
---
- MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo jd9365da_wxga_dsi_vdo"
30c30
+ MTK_SECURITY_SW_SUPPORT := no
---
- MTK_SECURITY_SW_SUPPORT := yes
34,44d33
+ MTK_DISABLE_GZ := yes
+ MTK_LK_NO_DISPLAY := yes
+ ifeq ("$(TARGET_BUILD_VARIANT)", "user")
+ MTK_LK_LOG_DISABLE := yes
+ LK_WITH_SMP := yes
+ MTK_DM_VERITY_OFF := yes
+ override DEBUG := 0
+ endif
+ ifeq ("$(TARGET_BUILD_VARIANT)", "userdebug")
+ override DEBUG := 1
+ endif
```

TF2.8:

```
vendor/mediatek/proprietary/trustzone/tf-a-
2.8/plat/mediatek/project/auto8676p1_64_bsp/platform.mk
```

```
MTK_SOC := mt6897
+CONFIG_MTK_GZ := n
+CONFIG_MTK_GZ_SECIO := n
+CONFIG_MTK_GZ_SMC := n
+LOG_LEVEL := 0
include plat/mediatek/$(MTK_SOC)/platform_makefile.mk
```

Kernel Config:

```
kernel/kernel_device_modules-6.1/arch/arm64/configs/auto8676p1_64_defconfig
```

设置/调整成如下 config:

```
CONFIG_ARM_FFA_TRANSPORT=n
CONFIG_MTK_GZ_KREE=n
CONFIG_MTK_DISABLE_GZ=y
CONFIG_MTK_DRAM_LOG_STORE=n
CONFIG_MTK_SENSOR_FAST_PROBE=y
# CONFIG_MTK_DRAM_LOG_STORE_ADDR is not set
# CONFIG_MTK_DRAM_LOG_STORE_SIZE is not set
# CONFIG_MTK_SECURITY_SW_SUPPORT is not set
```

Ko Table:

```
device/mediateksample/auto8676p1_64_bsp/ko_order_table.csv
```

根据如下方法删除和添加 ko:

```
delete,mkp.ko//删除掉 ko_order_table.csv 对应 mkp.ko 这一行
delete,ffa_v10.ko
delete,cmdq-sec-drv.ko
delete,mcDrvModule-ffa.ko
delete,gz_trusty_mod.ko
delete,gz_ipc_mod.ko
delete,gz_irq_mod.ko
delete,gz_virtio_mod.ko
delete,gz_tz_system.ko
delete,gz_main_mod.ko
delete,gz_log_mod.ko
delete,tmem_ffa.ko
delete,mtk_sec_heap.ko
delete,sapu.ko
delete,log_store.ko
delete,sec.ko
delete,phy-mtk-pcie.ko
delete,pcie-mediatek-gen3.ko
add,teeperf.ko,isee-ffa.ko,mcDrvModule.ko,../../kernel_device_modules-
6.1/drivers/tee/gud/600/MobiCoreDriver/mcDrvModule.ko,ramdisk,Y,Y,user/userdebug/eng
//在 teeperf.ko 的后面, isee-ffa.ko 的前面, 插入增加 mcDrvModule.ko
add,,cfg80211.ko,phy-mtk-pcie.ko,../../kernel_device_modules-6.1/drivers/phy/mediatek/phy-
mtk-pcie.ko,vendor,Y,Y,user/userdebug/eng
```

```
//在 cfg80211.ko 的前面，插入增加 phy-mtk-pcie.ko
add,,cfg80211.ko,pcie-mediatek-gen3.ko,../kernel_device_modules-
6.1/drivers/pci/controller/pcie-mediatek-gen3.ko,vendor,Y,Y,user/userdebug/eng
//在 cfg80211.ko 的前面，插入增加 pcie-mediatek-gen3.ko
```

1.5 常见问题/故障排除

1.5.1 系统问题需要提供的日志

表 1-2. 系统问题需要提供的日志明细

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
Bootup	M	M	O	O	X	X	X
NE	M	X	M	M	X	X	X
KE	M	C	M	M	X	X	X
SWT	M	C	M	C	X	X	X
ANR	M	X	M	O	C	C	C
JE	M	X	M	X	X	X	X
IPO	M	M	O	O	X	X	X
Performance	M	M	O	X	M	M	C

M: 必须提供

O: 可以不提供，根据问题的分析情况再提供

C: 有条件可以提供或者根据问题的分析情况再提供

X: 表示通常情况下不需要提供

1.5.2 MT8676 平台 DB 分析指引

1.5.2.1 AEE 介绍

Android Exception Engine (AEE)，是 MediaTek 自主开发的抓取日志的机制。当系统发生异常时，它能够抓取异常信息发生前后一段时间的日志并打包压缩成 DB 文件，保存到 eMMC 对应的位置。

1.5.2.2 DB 保存路径

AEE DB 保存到如下路径中：

```
/data/aee_exp
/data/vendor/aee_exp
```

1.5.2.3 DB 的种类

JE	• Java Exception
NE	• Native Exception
KE	• Kernel Exception
EE	• External Exception
ANR	• Application Not Response
SWT	• Software Watchdog Timeout

图 1-7. DB 的种类

1.5.2.4 Exception 分类

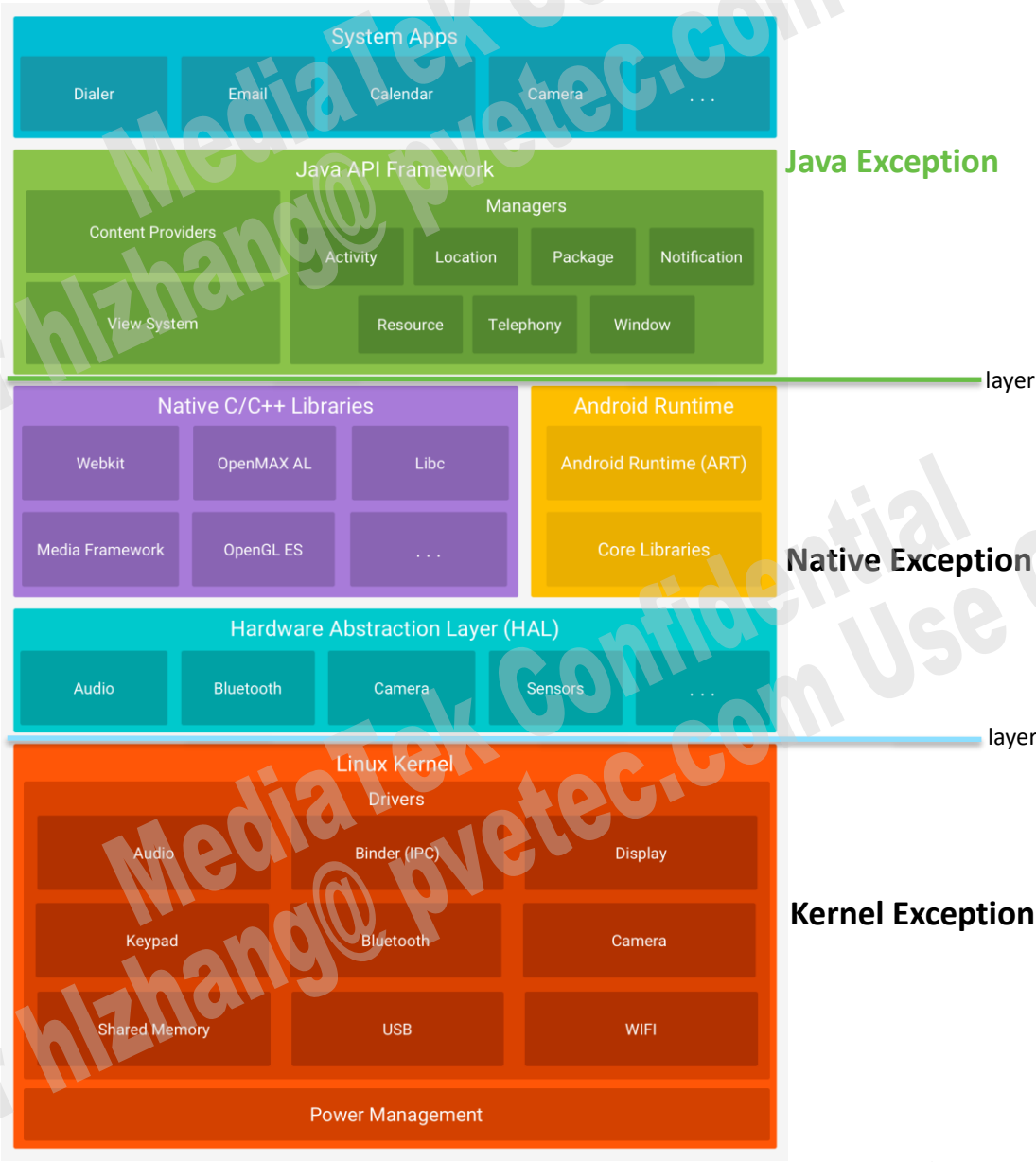


图 1-8. Exception 分类

1.5.2.5 GAT 解析 DB

1. 打开 GAT 工具。
2. 选择工具栏 Window→Open DBviewer。

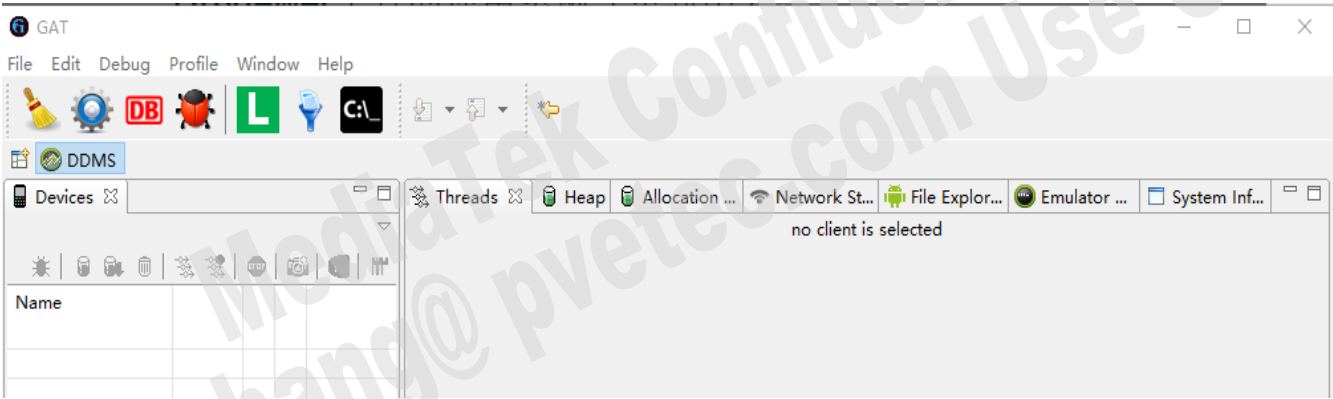


图 1-9. GAT Tool 主界面

3. DBViewer 弹出界面，如图 1-10 所示：



图 1-10. DBViewer 弹出界面

4. 选择 Select DB File，载入要解析的 DB 文件，如：db.fatal.00.KE.dbg。点击 Start，解析完成后，可看到解析出来的 DB Files。可在 dbg 文件同一路径下看到对应文件夹。



图 1-11. DB 解析后生成的文件

1.5.2.6 KE 解析范例

- KE: Kernel Exception
- KE DB 文件: db.fatal.00.KE, 如果发生 KE, 一般会在/data/vendor/aee_exp 中生成 DB 文件。
- 解压 db.fatal.00.KE.dbg 得到 db.fatal.00.KE.DBG.DEC。
- 解压开的 DB 内容, 重点关注_exp_main.txt。SYS_KERNEL_LOG 信息最多, 除了异常信息之外还有异常之前的一些 kernel 日志; SYS_LAST_KMSG 记录重启前最后的日志。

 _exp_main.txt	2020/4/14 17:19
 INTERRUPT_NUM_NAME_MAPS	2020/4/14 17:19
 out.json	2020/4/14 17:19
 PROC_CUR_TSK	2020/4/14 17:19
 SYS_KERNEL_LOG_RAW	2020/4/14 17:19
 SYS_LAST_KMSG	2020/4/14 17:19
 SYS_MINI_RDUMP	2020/4/14 17:19
 SYS_RAMCONSOLE_RAW	2020/4/14 17:19
 SYS_REBOOT_REASON	2020/4/14 17:19
 SYS_VERSION_INFO	2020/4/14 17:19
 ZAAE_LOG	2020/4/14 17:19
 ZZ_INTERNAL	2020/4/14 17:19

图 1-12. KE DB 解压后生成的文件

- 从_exp_main.txt 可知该 KE 是 USBNET 模块导致, 如图 1-13 所示:

```
Exception Class: Kernel (KE)
Exception Type: WDT timeout and Hardware reboot
PC is at [<ffffffc0007e7948>] skb_put+0xe4/0x108

Current Executing Process:
[khubd, 50][kthreadd, 2]

Backtrace:
[<ffffffc0009658f0>] __do_kernel_fault.part.5+0x70/0x84
[<ffffffc0000942d4>] do_bad_area+0x90/0x94
[<ffffffc000094320>] do_translation_fault+0x30/0x4c
[<ffffffc0000813f8>] do_mem_abort+0x38/0x98
[<ffffffc000083c58>] ell_da+0x1c/0x88
[<ffffffc00067ec64>] rx_complete+0x28/0x1ec
[<ffffffc000688d7c>] __usb_hcd_giveback_urb+0x58/0xf4
[<ffffffc000689448>] usb_hcd_giveback_urb+0x38/0x104
[<ffffffc00061de98>] musb_giveback+0xb8/0x12c
[<ffffffc00062373c>] musb_advance_schedule+0x270/0x754
[<ffffffc000625138>] musb_cleanup_urb.isra.9+0x118/0x524
[<ffffffc000625d18>] musb_urb_dequeue+0x3a8/0x3b4
[<ffffffc000689e94>] usb_hcd_unlink_urb+0x74/0xd8
[<ffffffc00068b930>] usb_unlink_urb+0x20/0x44
[<ffffffc00067d4c4>] unlink_urbs.isra.17+0x9c/0xc8
[<ffffffc00067d5ac>] usbnet_terminate_urbs+0x7c/0x11c
[<ffffffc00067de14>] usbnet_stop+0xe0/0x15c
```

图 1-13. USBNET 模块导致 KE

- 关于 KE 问题，需要提供的调试信息如表 1-3 所示：

表 1-3. KE 问题需要提供的调试信息

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
KE	M	C	M	M	X	X	X

注：vmlinux Symbol 信息位于 out/target/product/\$proj/obj/KERNEL_OBJ/vmlinux。

1.5.2.7 SWT 解析范例

SWT: Software Watchdog Timeout

SWT DB 文件: db.fatal.01.SWT

解压 db.fatal.01.SWT 得到 db.fatal.01.SWT.dbg.DEC。

解压开的 DB 内容，重点关注_exp_main.txt，如图 1-14 所示：

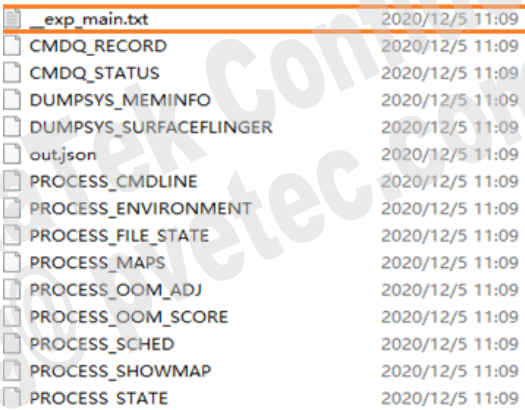


图 1-14. SWT DB 解压后生成文件

- 查看 exp_main.txt，system_server 发生异常导致 SWT，如图 1-15 所示：

```
$** ** Fatal **
Build Info: 'L1.MP13:B200_V1.0.0_320.240_1_V0.0.2_20181128_R:mt6735:S01
Flavor Info: 'None'
Exception Log Time:[Thu Dec 3 14:54:21 CST 2020] [128.466366]

Exception Class: SWT
Exception Type: system_server_watchdog

Current Executing Process:
system_server

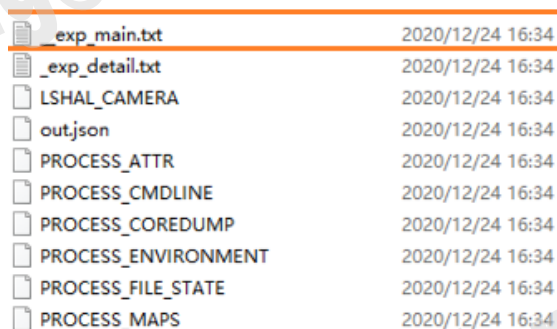
Backtrace:
Process: system_server
Subject: surfaceflinger hang.
Build: H_DiDi_B200_V3.1_20201202_R

$** ** Fatal **
```

图 1-15. system_server 发生异常导致 SWT

1.5.2.8 NE 解析范例

- NE: Native Exception
- NE DB 文件: db.fatal.NE
- 解压 db.fatal.00.NE.dbg 得到 db.fatal.00.NE.dbg.DEC。
- 解压开的 DB 内容，重点关注_exp_main.txt，如图 1-16 所示：



_exp_main.txt	2020/12/24 16:34
_exp_detail.txt	2020/12/24 16:34
LSHAL_CAMERA	2020/12/24 16:34
out.json	2020/12/24 16:34
PROCESS_ATTR	2020/12/24 16:34
PROCESS_CMDLINE	2020/12/24 16:34
PROCESS_COREDUMP	2020/12/24 16:34
PROCESS_ENVIRONMENT	2020/12/24 16:34
PROCESS_FILE_STATE	2020/12/24 16:34
PROCESS_MAPS	2020/12/24 16:34

图 1-16. NE DB 解压后生成的文件

- 查看 exp_main.txt，systemui 发生异常导致 NE，如图 1-17 所示：

Exception Class: Native (NE)
Exception Type: SIGABRT

Current Executing Process:

pid: 1271, tid: 1271
com.android.systemui

Backtrace:

```
#00 pc 00000000000821a0 /apex/com.android.runtime/lib64/bionic/libc.so (abort+176)
#01 pc 0000000000004ff4 /system/bin/app_process64 (art::SignalChain::Handler(int, siginfo*, void*)+1132)
#02 pc 0000000000000638 [vdso:00000072cab80000] (__kernel_rt_sigreturn)
#03 pc 000000000000cd6a4 /apex/com.android.runtime/lib64/bionic/libc.so (_epoll_pwait+4)
#04 pc 0000000000017da8 /system/lib64/libutils.so (android::Looper::pollinner(int)+144)
#05 pc 00000000000017c78 /system/lib64/libutils.so (android::Looper::pollOnce(int, int*, int*, void**) +56)
#06 pc 00000000001392bc /system/lib64/libandroid_runtime.so (android::android_os_MessageQueue_nativePollO
#07 pc 000000000002d7afc /system/framework/arm64/boot-framework.oat (art_jni_trampoline+140)
#08 pc 000000000007f9e04 /system/framework/arm64/boot-framework.oat (android.os.MessageQueue.next+228)
#09 pc 000000000007f6c08 /system/framework/arm64/boot-framework.oat (android.os.Looper.loop+680)
#10 pc 0000000000057dlcc /system/framework/arm64/boot-framework.oat (android.app.ActivityThread.main+812)
#11 pc 00000000001365b8 /apex/com.android.runtime/lib64/libart.so (art_quick_invoke_static_stub+568)
#12 pc 00000000001450cc /apex/com.android.runtime/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, u
#13 pc 000000000004b1190 /apex/com.android.runtime/lib64/libart.so (art::(anonymous namespace)::InvokeWith
#14 pc 000000000004b2bd8 /apex/com.android.runtime/lib64/libart.so (art::InvokeMethod(art::ScopedObjectAcc
#15 pc 0000000000043e928 /apex/com.android.runtime/lib64/libart.so (art::Method_invoke(_JNIEnv*, _jobject*
#16 pc 00000000000c5c34 /system/framework/arm64/boot.oat (art_jni_trampoline+180)
#17 pc 00000000000a78218 /system/framework/arm64/boot-framework.oat (com.android.internal.os.RuntimeInit$M
#18 pc 00000000000a801e4 /system/framework/arm64/boot-framework.oat (com.android.internal.os.ZygoteInit.ma
#19 pc 00000000001365b8 /apex/com.android.runtime/lib64/libart.so (art_quick_invoke_static_stub+568)
#20 pc 00000000001450cc /apex/com.android.runtime/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, u
#21 pc 000000000004b1190 /apex/com.android.runtime/lib64/libart.so (art::(anonymous namespace)::InvokeWith
#22 pc 000000000004b0df0 /apex/com.android.runtime/lib64/libart.so (art::InvokeWithVarArgs(art::ScopedObje
#23 pc 000000000003bba30 /apex/com.android.runtime/lib64/libart.so (art::JNI::CallStaticVoidMethodV(_JNIEnv
#24 pc 00000000000c058c /system/lib64/libandroid_runtime.so (_JNIEnv::CallStaticVoidMethod(_jclass*, _jme
#25 pc 00000000000c34ac /system/lib64/libandroid_runtime.so (android::AndroidRuntime::start(char const*,
#26 pc 00000000000034e0 /system/bin/app_process64 (main+1168)
#27 pc 000000000007d86c /apex/com.android.runtime/lib64/bionic/libc.so (__libc_init+108)
```

*** Exception ***

图 1-17. systemui 发生异常导致 NE

- NE 问题，需要提供的调试信息如表 1-4 所示：

表 1-4. NE 问题需要提供的调试信息

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
NE	M	X	M	M	X	X	X

注：NE Symbol 信息位于 out/target/product/\$proj/symbols。

1.5.2.9 JE 解析范例

- JE: Java Exception
- JE DB 文件: db.fatal.00.JE
- 解压 db.fatal.00.JE.dbg 得到 db.fatal.00.JE.dbg.DEC。
- 解压开的 DB 内容，重点关注_exp_main.txt，如图 1-18 所示：

 exp_main.txt	2020/11/25 15:21
 DUMPSYS_ACTIVITY	2020/11/25 15:21
 DUMPSYS_DBINFO	2020/11/25 15:21
 DUMPSYS_MEMINFO	2020/11/25 15:21
 DUMPSYS_SURFACEFLINGER	2020/11/25 15:21
 DUMPSYS_WINDOW	2020/11/25 15:21
 out.json	2020/11/25 15:21
 PROCESS_CMDLINE	2020/11/25 15:21
 PROCESS_ENVIRONMENT	2020/11/25 15:21
 PROCESS_FILE_STATE	2020/11/25 15:21
 PROCESS_MAPS	2020/11/25 15:21

图1-18. JE DB 解压后生成的文件

- 查看 exp_main.txt，animation 异常导致 system_server crash 引起 JE，如图 1-19 所示：

```
Exception Class: Java (JE)
Exception Type: system_server_crash
```

```
Current Executing Process:
system_server
```

```
Backtrace:
Process: system_server
Build: NEUSOFT/full_fsllg_qj_2wk/fsllg_qj_2wk:5.1/LMY47D/1605154054:user/test-keys
```

```
java.lang.IllegalArgumentException: The Path cannot have discontinuity in the X axis.
    at android.view.animation.PathInterpolator.initPath(PathInterpolator.java:177)
    at android.view.animation.PathInterpolator.initCubic(PathInterpolator.java:154)
    at android.view.animation.PathInterpolator.parseInterpolatorFromTypeArray(PathInterpolat
    at android.view.animation.PathInterpolator.<init>(PathInterpolator.java:100)
    at android.view.animation.AnimationUtils.createInterpolatorFromXml(AnimationUtils.java:
    at android.view.animation.AnimationUtils.loadInterpolator(AnimationUtils.java:283)
    at com.android.internal.policy.impl.PhoneWindow$DecorView.<init>(PhoneWindow.java:2331)
    at com.android.internal.policy.impl.PhoneWindow.generateDecor(PhoneWindow.java:3465)
    at com.android.internal.policy.impl.PhoneWindow.installDecor(PhoneWindow.java:3851)
    at com.android.internal.policy.impl.PhoneWindow setContentView(PhoneWindow.java:411)
    at com.android.internal.app.AlertController.installContent(AlertController.java:240)
    at android.app.AlertDialog.onCreate(AlertDialog.java:356)
    at android.app.ProgressDialog.onCreate(ProgressDialog.java:198)
    at android.app.Dialog.dispatchOnCreate(Dialog.java:386)
    at android.app.Dialog.show(Dialog.java:287)
```

图1-19. Animation 异常导致 system_server crash 引起 JE

1.5.2.10 EE 解析范例

- EE: External Exception，如 modem、connectivity 发生的 exception
- EE DB 文件：db.02.EE
- 解压 db.02.EE.dbg 得到 db.02.EE.dbg.DEC。
- 解压后的 DB 内容，重点关注 exp_main.txt。
- MD 发生了 ASSERT 错误，如图 1-20 所示：

```
Exception Log Time:[Sat Dec 6 13:35:09 CST 2023] [11633.650000]
Exception Class: External (EE)
Exception Type: modem
Current Executing Process:
mdl:
[ASSERT] file:RXAGC_C1_Main.c @FMC line:867
```

图1-20. 触发了MD ASSERT

1.5.2.11 ANR 解析范例

- ANR: Application Not Response，即应用无响应，指某个应用程序的主线程在一段时间内没有做完某件事情。
- ANR DB 文件：db.00.ANR
- 解压 db.00.ANR.dbg 得到 db.00.ANR.dbg.DEC。
- 解压后的 DB 内容，重点关注_exp_main.txt 与 SWT_JBT_TRACES。
- System UI 发生 ANR，如图 1-21 所示：

```
Exception Class: ANR
Exception Type: system_app_anr

Current Executing Process:
com.android.systemui
com.android.systemui v22 (5.1-1605618285)

Backtrace:
Process: com.android.systemui
Flags: 0x40c83e0d
Package: com.android.systemui v22 (5.1-1605618285)
Subject: Broadcast of Intent { act=android.intent.action.TIME_TICK flg=0x50000114 (has extras) }
```

图1-21. System UI 发生 ANR

- 关于 ANR 问题，需要提供的调试信息如表 1-5 所示：

表 1-5. ANR 问题需要提供的调试信息

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
ANR	M	X	M	O	C	C	C

- 如何分析 ANR 问题？
可以通过查看 androidlog 来迅速定位 ANR 发生的位置并获取一些必要的资讯。（小技巧：可以通过搜索“ANR”快速定位，多一个空格，过滤掉一些干扰的日志信息。）

```

11-22 15:31:27.520 1785 1800 E ActivityManager: ANR in com.amazon.aiv.eu (com.amazon.aiv.eu/com.amazon.ignition.IgnitionActivity)
11-22 15:31:27.520 1785 1800 E ActivityManager: PID: 6229
11-22 15:31:27.520 1785 1800 E ActivityManager: Reason: Input dispatching timed out (Waiting because no window has focus but there is a f
11-22 15:31:27.520 1785 1800 E ActivityManager: Load: 32.24 / 31.91 / 31.88
11-22 15:31:27.520 1785 1800 E ActivityManager: CPU usage from 0ms to 11679ms later (2017-11-22 15:31:15.731 to 2017-11-22 15:31:27.411):
11-22 15:31:27.520 1785 1800 E ActivityManager: 45% 6229/com.amazon.aiv.eu: 34% user + 10% kernel / faults: 48921 minor 943 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 26% 1201/surfaceflinger: 8.1% user + 18% kernel / faults: 1156 minor 13 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 23% 1785/system_server: 14% user + 8.6% kernel / faults: 9885 minor 487 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 18% 1290/VDEC_0: 0% user + 18% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 13% 650/mmcqd/0: 0% user + 13% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 12% 430/kswapd0: 0% user + 12% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 9.9% 1465/dtv_svc: 3.2% user + 6.6% kernel / faults: 1717 minor 67 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 8.2% 2684/com.mediatek.tvinput: 6% user + 2.1% kernel / faults: 2755 minor 25 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 7.1% 11637/kworker/u9:8: 0% user + 7.1% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 6.9% 1595/MLVdo_thread: 0% user + 6.9% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 6.7% 30988/kworker/u9:0: 0% user + 6.7% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 6.6% 2968/org.droidtv.playtv: 5% user + 1.6% kernel / faults: 3315 minor 62 major

```

图 1-22. 如何通过日志快速定位 ANR 问题

如果 ANR 发生了，对应的应用会收到 SIGQUIT 异常终止信号，dalvik 虚拟机就会自动在 /data/anr/ 目录下生成 trace.txt 文件，这个文件记录了 ANR 发生时刻系统各线程的执行状态。

1.5.3 Memory Leak 问题处理方法

1.5.3.1 Memory Leak 简单介绍

判断是否有 memory leak 的关键信息是 memory 占用是否一直在涨。

Memory Leak 造成的原因是某个被配置 (allocated) 的记忆体无法被参照 (referenced)，也无法被释放 (released)；那块被配置的记忆体就有如记忆体孤儿般，无法被系统再使用。

1.5.3.2 查看 Memory 的命令

- dumsys meminfo 查看 Android 系统详细的 memory 分布。
- cat /proc/meminfo 查看总体的 memory 分布。

通过命令或者 bat 脚本，在复制问题的同时不断的在后台 dump memory 信息。

1.5.3.3 判断 Kernel Memory Leak 的方式

输入命令：cat /proc/meminfo:

```

MemTotal:      965616 kB
MemFree:       114640 kB
MemAvailable:  491072 kB
Buffers:       7224 kB
Cached:        393032 kB
Shmem:         1184 kB
Slab:          55916 kB
SReclaimable:  18788 kB
SUnreclaim:    37128 kB
KernelStack:   12592 kB
PageTables:    17876 kB
VmallocUsed:   0 kB

```

图 1-23. cat /proc/meminfo 日志

- 如果 sunreclaim 这个值很大且持续增长，基本可以确定 kernel Memory Leak 发生了，如图 1-23 所示。
- 在 kernel 日志中，如果发生了 OOM (Out of Memory)，也会 dump 出来这部分的 memory。如果看到 sunreclaim 很大且持续增长，基本上也可以判断是有 kernel Memory Leak，如图 1-24 所示：

```

[147842.604198] slab_reclaimable:12762 slab_unreclaimable:119398
.....
[151361.050385] slab_reclaimable:8474 slab_unreclaimable:122915
.....
[153699.412832] slab_reclaimable:7833 slab_unreclaimable:124784

```

图 1-24. Kernel memory leak 日志

1.5.4 WDT 简单介绍

WDT 全称是 Watchdog Timer，指的是看门狗模块，看门狗其实就是一个可以在一定时间内被复位的计数器。当看门狗启动后，计数器开始自动计数，经过一定时间，如果计数没有被复位，计数器达到指定数值，就会发出复位信号，很多设备包括 CPU 接到这个信号会复位重启（俗称“被狗咬”）。为了保证看门狗不发出复位信号，就需要在看门狗允许的时间间隔内对看门狗计数器清零（俗称“喂狗”），计数器重新计数。如果系统正常并保证按时“喂狗”，那么就相安无事。一旦程序故障卡死，没有“喂狗”，系统会“被咬”继而复位。

1.5.4.1 SWT

SystemService 是 Android 的一个核心进程，它为 APP 运行提供了核心的服务。如果 SystemServer 的一些核心服务和重要线程卡住，就会导致相应的功能异常，如手机发生 hang 机，输入无响应，无法启动 APP 等一些不正常的情况。而且，如果没有一种机制让这些服务复位的话，那么将严重影响客户体验。尤其是当前大多数手机把电池封装在手机里面，想拨电池重启都很难。

所以有必要在核心服务和核心线程卡住的时候，让系统有自动复位的机会。于是，Google 引入了 SystemServer watchdog 机制。这个机制可以监控核心服务和核心线程是否卡住。

1.5.4.2 HWT

WDT 第 1 阶段超时将触发 FIQ，此时 CPU 收到 FIQ 并处理，最后调用 BUG() 走正常的 kernel exception 流程，生成的 DB 类型为 HWT (Hardware Watchdog Timeout)。

1.5.4.3 HW Reboot

WDT 第 1 阶段超时将触发 FIQ，如果 CPU 没有响应或 CPU 响应了但没有在第 2 阶段超时时间内完成重启，就会导致第 2 阶段超时，WDT 发出复位信号复位整个系统，生成的 DB 类型为 HW reboot。

1.5.5 性能模式

1.5.5.1 性能模式简单介绍

Performance Mode（性能模式）是一种在移动设备、笔记本电脑和台式机等设备上提高系统性能的配置选项。在性能模式下，设备会优先使用更高的 CPU 频率、更大的内存缓存、更高的屏幕刷新率和更高的功率消耗等，以提供更出色的系统性能和响应速度。这种模式通常是在需要高性能运行的时候使用。

1.5.5.2 adb 设置性能模式

不同的 IC 目录不一样，但是设置方法一样。

CPU:

```
ls /sys/devices/system/cpu/cpufreq/这个目录下有几个簇
echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy7/scaling_governor
```

或者:

```
adb shell "echo 0 1900000 1900000 > /proc/cpudvfs/cpufreq_debug"
adb shell "echo 4 2800000 2800000 > /proc/cpudvfs/cpufreq_debug"
adb shell "echo 7 2900000 2900000 > /proc/cpudvfs/cpufreq_debug"
```

first number (1900000) means floor and second number means ceiling.

GPU Set:

```
adb shell "echo 0 > /proc/gpufreqv2/fix_target_opp_index"
```

GPU Query:

```
adb shell "cat /proc/gpufreqv2/gpufreq_status"
```

DDR:

Set DDR frequency

```
adb shell "echo 0 > /sys/kernel/helio-dvfsrc/dvfsrc_force_vcore_dvfs_opp"
```

Check DDR frequency

```
adb shell "cat /sys/kernel/helio-dvfsrc/dvfsrc_dump | grep bps"
```

Thermal disable:

```
adb shell "thermal_intf apply disable_throttling.conf"
```

1.5.5.3 在代码中修改开机自启动性能模式

在/device/mediatek/mtxx/init.mtxxx.rc 里做如下修改:

```
# switch to sched-dvfs
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor "schedutil"
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor "schedutil"
write /sys/class/net/p2p0/queues/rx-0/rps_cpus "fe"
write /sys/class/net/p2p0/queues/rx-1/rps_cpus "fe"
write /sys/class/net/p2p0/queues/rx-2/rps_cpus "fe"
write /sys/class/net/p2p0/queues/rx-3/rps_cpus "fe"
# stop bootprof
write /proc/bootprof 0
```

其中这两句

```
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor "schedutil"
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor "schedutil"
```

改为

```
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor "performance"
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor "performance"
```

1.5.6 CPU 插拔

您可以用如下指令来开关核。

```
adb shell "echo 1 > /sys/devices/system/cpu/cpu1/online"
adb shell "echo 1 > /sys/devices/system/cpu/cpu2/online"
adb shell "echo 1 > /sys/devices/system/cpu/cpu3/online"
```

echo 1 是打开, echo 0 是关闭。

1.5.7 如何抓取 systrace

1. 使用 adb 命令抓 trace, 10s 表示抓 10s, 最多可以抓 20s。

```
adb shell perfetto -o /data/misc/perfetto-traces/trace -t 10s sched freq idle am wm gfx view
```

2. 使用脚本抓 ftrace

01-catch.bat cpu sched 和 workqueue 也可以抓到。

3. 使用 android Q 以及 Q 之后自带的工具抓 trace。

使用命令打开 adb shell am start com.android.traceur/com.android.traceur.MainActivity, 然后将 CPU 参数设置为最大, 点击 record trace 开始录制异常状态, 一段时间之后关闭, 抓取 trace 下来, trace 保存在 /data/local/traces。

4. 从 trace 可以看到进程有没有进入 queue, 以及有没有被执行, 以及被安排到哪个 CPU。如果 workqueue 是没有绑核的, 只能靠系统根据 loading 调度, 程序可能在任何 CPU 上运行。

1.5.8 系统常见问题相关参考

内存泄漏专题分析: <https://online.mediatek.com/apps/quickstart/QS00086>

踩内存专题分析: <https://online.mediatek.com/apps/quickstart/QS00090>

死机问题快速分析: <http://online.mediatek.com/QuickStart/QS00044>

Memory: <http://online.mediatek.com/QuickStart/QS00227>

Performance 调试宝典: <https://online.mediatek.com/apps/quickstart/QS00288>

深入分析 Android native exception 框架: <http://online.mediatek.com/QuickStart/QS00038>

深入分析看门狗框架: <http://online.mediatek.com/QuickStart/QS00067>

Hang Detect 问题快速分析 <https://online.mediatek.com/apps/quickstart/QS00076>

深入分析 Linux kernel exception 框架 <http://online.mediatek.com/QuickStart/QS00073>

NE/KE 分析学习课程 <https://online.mediatek.com/apps/quickstart/QS00068>

NE/KE 分析报告解读 <https://online.mediatek.com/apps/quickstart/QS00089>

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。